# CS149 Assignment 1 实验报告：多核、SIMD、带宽与 GPU 对照

ShaneLiu04*

2026 年 10 月

**摘要**

Stanford CS149（Fall 2024）Assignment 1 “Analyzing Parallel Program Performance on a Quad-Core CPU” 用六个小程序让人亲手测出并行程序最常见的三类瓶颈：负载不均衡、SIMD 发散和内存带宽。本文完成了全部六个 program，并且多走了一步：把同样的三个问题搬到 GPU 上，看答案是否还成立。

实现全部用 C++17：`std::thread` 版 Mandelbrot、自写的 CS149 fake vector intrinsics、用 AVX2/AVX-512 intrinsics 手写的 ISPC gang（没有 ISPC 编译器）、Newton 法 sqrt、带 non-temporal store 的 saxpy，以及 K-means 的 profiling 与三级优化；GPU 部分用 CUDA 在 RTX 5090 上实现。所有 SIMD、多线程和 GPU 版本的结果都与标量参考逐位相等（编译时关闭了 FMA 合并）。计时取 3 轮独立实验的中位数，并给出最小–最大区间。

主要发现如下。负载均衡方面，连续块静态划分在 view 1 上用 12 线程只有 5.89 倍，换成动态分块后达到 11.1 倍。Mandelbrot 内核受依赖链延迟限制，所以一个物理核上的两个 SMT 线程几乎顶得上两个核；同样的道理，在单线程里交错推进两组向量（提高 ILP）能把 AVX-512 加速从 11.2 倍提到 18.3 倍，超过 16 lane 的宽度。SIMD 发散方面，Newton sqrt 在 “每 W 个元素里有 1 个慢元素” 的输入下，AVX2 比标量还慢（0.72 倍）；AVX-512 在三种发散程度不同的输入上耗时完全相同。带宽方面，saxpy 单线程只能拿到 23 GB/s 的 DRAM 带宽，12 线程为 173 GB/s；streaming store 省掉 write-allocate 后再高 3%–33%，跨 socket 访问则降 30%。K-means 只并行化 `computeAssignments` 时被 Amdahl 定律限制在 2.1 倍，三个阶段全部并行、再做循环交换和向量化后达到 11.7 倍。最后，同一个 Mandelbrot 在 RTX 5090 上比 12 核 AVX-512 双向量交错版还快 60–100 倍；但对 saxpy 来说，只要数据需要跨 PCIe，GPU 的端到端吞吐（52 GB/s）就比 CPU 低。每一节最后附有面试问法与常见错答。

## 目录

**1 作业要求与本文实现** **3**

**2 实验平台与方法** **3**

2.1 硬件与软件 . . . . . 4

2.2 正确性 . . . . . 4

2.3 计时 . . . . . 4

**3 Program 1：线程与负载均衡** **4**

3.1 工作量的空间分布 . . . . . 5

3.2 三种调度的加速比 . . . . . 5

*GitHub: `https://github.com/ShaneLiu04`。代码与原始指标：`https://gitee.com/liu-xingyan04/mlsys-infra`。本文基于公开课程材料独立完成，与课程团队无隶属关系。

3.3 动态分块的粒度 . . . . . 6
3.4 SMT：两个硬件线程能顶几个核 . . . . . 7

**4 Program 2：向量 intrinsics 与 lane 利用率 8**
4.1 clampedExp . . . . . 8
4.2 arraySum . . . . . 9

**5 Program 3：SIMD gang 与 task 9**
5.1 手写 gang：AVX2、AVX-512 与双向量交错 . . . . . 9
5.2 task 数量 . . . . . 10
5.3 把 SIMD、ILP、多核和 GPU 乘起来 . . . . . 11
5.4 一个测量陷阱：worker 放在哪个 CPU 上 . . . . . 11

**6 Program 4：Newton sqrt 与最好、最坏输入 12**

**7 Program 5：saxpy 与内存带宽 13**
7.1 带宽随工作集的变化 . . . . . 14
7.2 write-allocate 与 streaming store . . . . . 14
7.3 线程扩展与 NUMA . . . . . 14

**8 Program 6：K-means 的 profiling 与优化 15**

**9 GPU 对照：同样的三个问题在 SIMT 上 16**
9.1 Mandelbrot：warp 效率与 occupancy . . . . . 16
9.2 sqrt：GPU 上发散为什么“不见了” . . . . . 16
9.3 saxpy：GPU 赢在显存，输在 PCIe . . . . . 17
9.4 K-means 的 assignment kernel：合并访存 . . . . . 18
9.5 为什么 GPU 比 12 核 AVX-512 快 60–100 倍 . . . . . 18

**10 仓库原有 Python 版本的回顾 19**

**11 面试专题 19**
11.1 20 秒版本 . . . . . 19
11.2 追问与答案 . . . . . 20
11.3 STAR 故事 . . . . . 21

**12 局限 21**

**13 结论 22**

# 1 作业要求与本文实现

为什么一个程序开了 12 个线程，却只快了不到 6 倍？CS149 [1] Assignment 1 想让学生在一台四核 CPU 上亲手找到答案。它准备了六个 program，每个都把一种瓶颈放大到肉眼可见。表 1 列出官方六个 program 想回答的问题，以及本文对应的实现。

官方 starter 依赖 ISPC [2] 编译器（一个把 C 风格代码编译成 SIMD 指令的编译器）。我没有用它，而是用 AVX2/AVX-512 intrinsics [13] 手写 ISPC 的 gang，用一个靠原子计数器分发任务的线程池代替 ISPC 的 task。这样做麻烦一些，但每条向量指令都摆在眼前，后面分析为什么加速比不到理论值时会方便很多。同一份代码在 Windows 本机（MinGW g++ 13）和 Linux 服务器（g++ 11）上编译运行。

表 1: 官方 program 与本文实现

| 官方 program | 官方要回答的问题 | 本文实现（`cpp/`） |
|---|---|---|
| prog1 Mandelbrot threads | 线程数与加速比；为什么连续块划分不线性；如何划分到 7–8 倍 | 连续块、隔行、动态分块三种调度，1–24 线程，SMT [6] 与 NUMA 布局，每线程计时 |
| prog2 vector intrinsics | 用 fake intrinsics 写 `clampedExp` 与 `arraySum`，统计 lane 利用率随宽度的变化 | 自写 `cs149intrin.h`（同名 API 与 logger），W=2/4/8/16，5 种指数分布 |
| prog3 ISPC Mandelbrot | gang 加速比为什么不到 8；task 数量的选择 | AVX2 8 lane、AVX-512 16 lane、AVX-512 双向量交错；task 数 1–1200 扫描 |
| prog4 sqrt | 构造让 ISPC 加速最好和最差的输入 | 标量、AVX2、AVX-512、SIMD+ 线程；random / all_slow / all_one / one_slow_8 / one_slow_16 |
| prog5 saxpy | 为什么多核加速有限；写回带来的额外流量 | 标量、SIMD、streaming store、纯读四个内核，工作集 12 KB–1.6 GB，NUMA |
| prog6 K-means [12] | 用计时找瓶颈并优化 | 官方结构的 baseline 与 v1/v2/v3 三级优化，分阶段计时 |

另外有两部分是作业之外的补充。第 9 节把 prog3、prog4、prog5、prog6 的核心内核写成 CUDA，在 RTX 5090 上检验 “同一个问题换到 SIMT [9]（GPU 那种由硬件把 32 个线程打包成一组执行的模型）上，答案是否一样”。第 10 节简要回顾仓库里原有的 Python 版本，说明为什么解释器版本的数字不能用来回答作业问题。

# 2 实验平台与方法

后面每一节都要比较几个百分点到几倍的差别。要让这些比较站得住，需要三个前提：用的核是干净的，结果是对的，计时是稳的。这一节逐一说明本文怎样保证这三点。

### 2.1 硬件与软件

表 2: 实验平台

| | 服务器（主平台） | 本机（四核对照） |
|---|---|---|
| CPU | 2 × Intel Xeon Gold 6459C（Sapphire Rapids），每路 32 核，SMT 2 | Intel Ice Lake 服务器核（虚拟机），4 核 8 线程 |
| 缓存 | 每核 L1d 48 KB、L2 2 MB；每路 L3 60 MB | 每核 L1d 48 KB、L2 1.25 MB；L3 57 MB |
| 可用 CPU | cgroup 配额 16 个 CPU；本文固定绑在 socket 0 的 12 个空闲物理核上 | 4 物理核，SMT 兄弟为 $(2k, 2k{+}1)$ |
| 内存 | 754 GB，2 个 NUMA 节点 | 32 GB |
| ISA | AVX2、AVX-512（含 BF16/FP16）、AMX | AVX2、AVX-512 |
| GPU | NVIDIA RTX 5090（sm_120，170 SM，32 GB GDDR7，512 bit，96 MB L2） | 无 |
| 编译器 | g++ 11.4，CUDA 12.8 | MinGW-w64 g++ 13.2 |

服务器是多租户宿主机。容器能看到 128 个逻辑 CPU，但 cgroup 只给了 16 个 CPU 的时间配额，而且其他租户会占满个别核。所以开始实验前，我先用 `/proc/stat` 采样了 3 秒，发现 CPU 15 和 CPU 20 被其他租户占到 99%。之后所有实验都固定使用 socket 0 上的 {0,1,2,3,4,6,10,11,14,16,19,23} 这 12 个物理核。SMT 实验使用它们的兄弟核 $c{+}64$，NUMA 实验使用 socket 1 上的 12 个空闲核。拓扑由 `lscpu -e`（Linux）和 `GetLogicalProcessorInformationEx`（Windows，本文的 `topology.exe`）确认。

### 2.2 正确性

SIMD 版本算错了也可能跑得很快，所以每个加速比都必须建立在 “结果完全一致” 之上。所有程序用 `-O3 -march=native -ffp-contract=off` 编译。其中 `-ffp-contract=off` 禁止编译器把 $a \cdot b + c$ 合并成 FMA（融合乘加，只舍入一次）。这样，手写的 SIMD 内核只要逐个操作照搬标量代码的运算顺序（例如先算 `2*zr` 再乘 `zi`），结果就与标量版本逐位相等。比较 $|z|^2 > 4$ 时用 `_CMP_NGT_UQ`（“不大于”，遇到 NaN 时为真），而不是 `_CMP_LE_OQ`，这样才与标量 `!(x>4)` 的 NaN 语义一致。

每个程序在每次测量后都检查结果,任何不一致都会让进程以非零码退出;`tests/test_cpp.py` 用小规模输入跑全部六个 program。GPU 版用 `nvcc -fmad=false` 编译，host 端参考使用相同的运算顺序，Mandelbrot 与 sqrt 同样逐位相等（mismatch = 0）。

### 2.3 计时

每个配置取 $N$ 次运行的最小值（prog1 和 prog3 为 7 次，prog4 为 3 次），这与官方 starter 的做法相同。整个实验在服务器上独立重复 3 轮，文中数字是 3 轮的中位数，图中误差棒是最小–最大值。prog5 用常驻的自旋线程池，每个尺寸至少运行 0.2 s。还有一点要提前说明：服务器的单线程 turbo 频率高于全核频率，所以多线程加速比天然会低于线性，第 3 节会看到这一点。

## 3 Program 1：线程与负载均衡

prog1 把 Mandelbrot 图像按行分给多个线程。直觉上 12 个线程应该快 12 倍，但官方的连续块划分离这个数字很远。这一节先看工作量在图像上怎么分布，再比较三种调度方式，最后讨论两个容易被忽略的因素：分块粒度和超线程。

### 3.1 工作量的空间分布

Mandelbrot 的每个像素要迭代到逃逸或达到上限为止，所以不同像素的代价可以差上百倍。图 1 左侧是两个视图的迭代数，右侧是每行的总工作量（迭代数 +1，因为最后一次判断也要算）。view 1 的中间几行穿过 Mandelbrot 集的主体，那里每个像素都跑满 256 次；边缘的行几乎立刻逃逸。

把 1200 行切成 $P$ 个连续块时，墙钟时间由最重的那一块决定。因此静态划分的加速比有一个上限：

$$S_{\text{block}} \le \frac{\sum_r w_r}{\max_k \sum_{r \in B_k} w_r} = \frac{P}{\max_k W_k / \bar{W}}. \tag{1}$$

图中标出了 $P = 4$ 与 $P = 12$ 时最重块与平均块的比值 max /mean。view 1 分别为 1.61 和 2.00，也就是说，连续块划分在 12 线程时最多只能快 6 倍。view 2 的比值是 1.49 和 1.97。

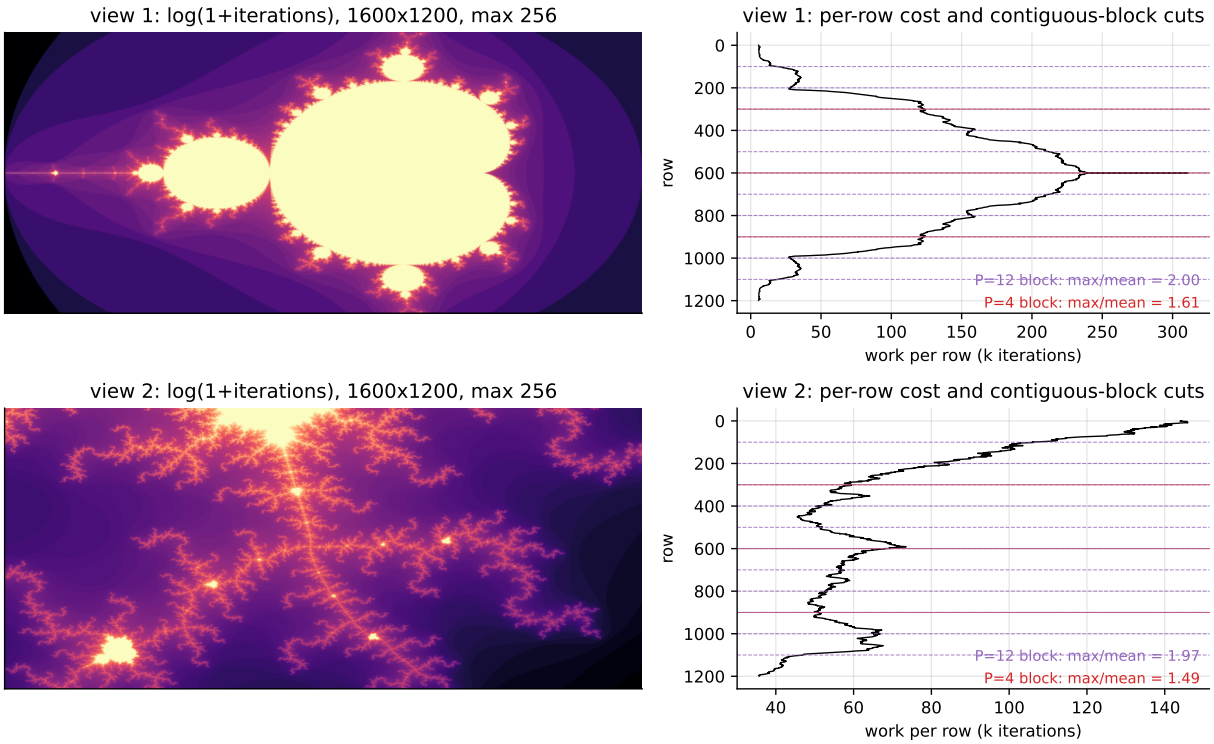



图 1: 左：两个视图的 log(1 + 迭代数)；右：每行工作量与 $P = 4$（实线）、$P = 12$（虚线）的连续块切分线。

### 3.2 三种调度的加速比

有了式 (1)，就可以预测三种调度方式的表现，再用图 2 的实测加速比来验证：

- 连续块（block）：view 1 在 3 线程时只有 1.64 倍，比 2 线程（1.99 倍）还低。原因是三等分后，中间那一块独揽了整个集合主体。12 线程时为 5.89 倍，已经贴近式 (1) 给出的 6 倍上限。
- 隔行（interleaved）：线程 $t$ 取第 $t, t+P, \ldots$ 行。相邻行的工作量很接近，隔行分配把它们打散到不同线程上，所以工作量几乎完全均衡。但它仍然是静态划分：只要有一个核被其他租户抢占，所有线程都得等它。
- 动态（dynamic，每次取 4 行）：线程从一个原子计数器上领任务，谁先做完谁多领。这样既均衡了工作量，也自动绕开了慢核。view 1 在 8 线程时达到 7.74 倍，12 线程时 11.1 倍。

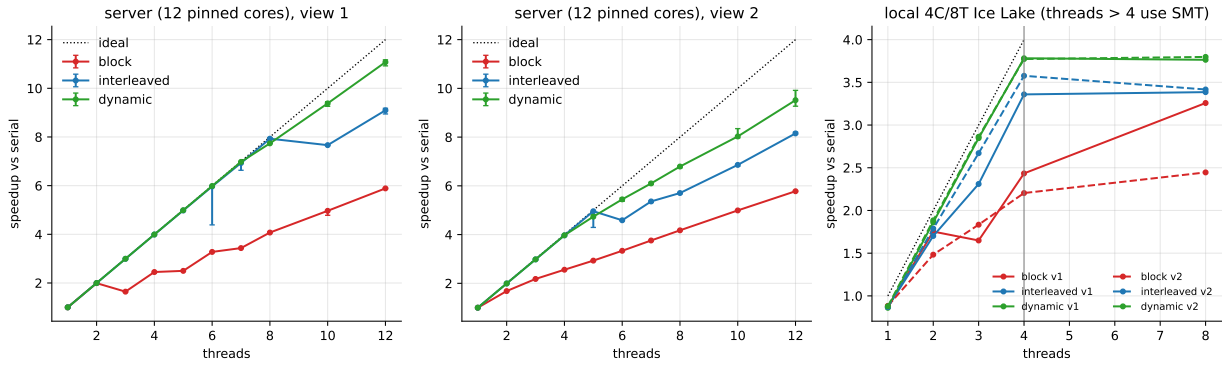



图 2: prog1 加速比。左、中：服务器 12 个绑定物理核，3 轮中位数与最小–最大区间；右：本机四核（4 线程以上使用 SMT）。

加速比曲线只告诉我们“不线性”，要看清为什么，得把每个线程单独拿出来看。图 3 是 8 线程时每个线程的计算时间。连续块划分下，0 号和 7 号线程几乎无事可做，3 号和 4 号线程决定了墙钟时间；隔行和动态划分下，8 个线程的时间几乎一样，墙钟时间约为连续块的一半。面试里被问到“为什么加线程不线性”，这张图就是最直接的回答：墙钟时间等于最慢线程的时间，而不是所有线程的平均时间。

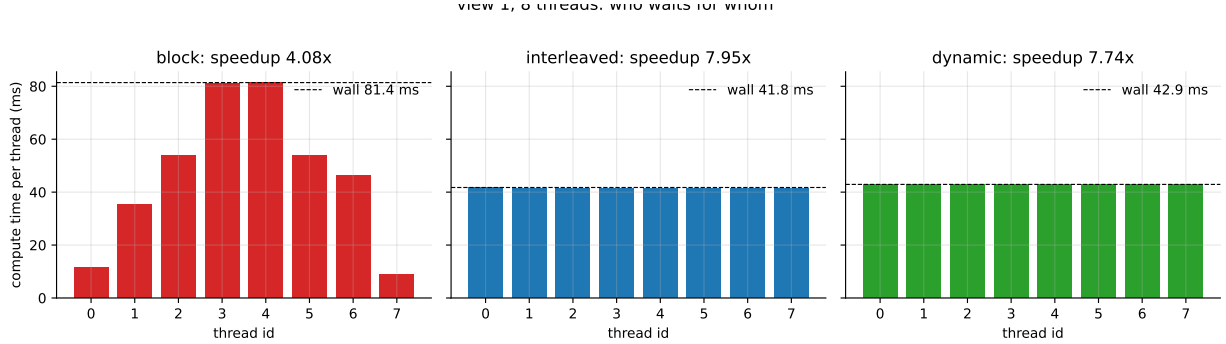



图 3: view 1、8 线程时每个线程的计算时间，虚线为墙钟时间。

## 3.3 动态分块的粒度

动态调度每次领多少行合适？领得太少，每次领取都要一次原子操作；领得太多，最后几块做不完时其他线程只能干等。图 4 扫描了每次领取的行数。chunk 为 1–16 行时加速比基本持平（12 线程 view 1 为 9.2–9.5 倍），从 32 行开始下降；chunk 增大到 100 行时每个线程只领一次，退化成连续块划分（5.9 倍）。曲线左侧之所以很平，是因为 Mandelbrot 每行约 0.3 ms，而一次原子操作只要几十纳秒，小粒度的开销可以忽略。

这里我踩到了一个测量上的坑。同样是 12 线程、chunk = 4，粒度扫描得到 9.3 倍，而图 2 的线程扫描得到 11.1 倍。3 轮实验都稳定地相差约 18%（墙钟 35.5 ms 对 30.0 ms）。两者唯一的区别是：线程扫描从 1 线程逐步加到 12 线程，跑到 12 线程时这些核已经忙了一段时间；粒度扫描则一开始就在 12 个刚从空闲中唤醒的核上运行。我推测原因是空闲核的频率爬升（服务器的 HWP 策略可能需要上百毫秒），但没有读取频率计数器验证。由此得到的教训是：比较两个配置时，它们必须在相同的“预热”条件下测量。本文同一张图内的数据都满足这一点。

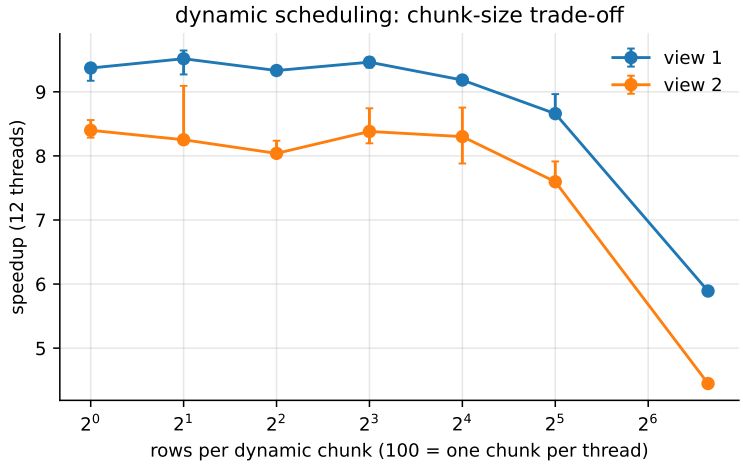



图 4: 动态调度的粒度：12 线程加速比随每次领取的行数变化。

### 3.4 SMT：两个硬件线程能顶几个核

官方作业还问了超线程（SMT，一个物理核同时运行两个硬件线程）对 Mandelbrot 有没有用。图 5 把同样数量的线程放在不同数量的物理核上，比较耗时。6 个线程放在 3 个物理核上（每核 2 个 SMT 线程）时，标量版耗时 62.5 ms；放在 6 个独立核上为 56.3 ms。换句话说，3 个核借助 SMT 完成了 6 个核约 90% 的工作。

为什么收益这么大？因为 Mandelbrot 的内层循环是一条依赖链：`zr*zr`、`zi*zi`、相加、比较、再更新 `zr` 和 `zi`，每一步都要等上一步的 4 周期乘法延迟。一个线程只能用掉执行端口的一小部分，第二个 SMT 线程的依赖链正好填进空闲的周期。所以 SMT 的收益取决于代码是受延迟限制还是受吞吐限制，并没有一个固定的 “20%–30%”。

图中紫色柱是反面验证。第 5 节的双向量交错版在单线程内就交错了两条独立的依赖链，已经填掉了一部分空闲周期，所以它从 SMT 得到的收益最少。从 6 个核挤到 3 个核（每核 2 个 SMT 线程）时，view 1 上标量版耗时增加 11%，AVX-512 版增加 3%，双向量交错版增加 17%；view 2 上交错版增加 45%。12 线程挤到 6 个核时也是如此：交错版慢 54%，单向量 AVX-512 慢 23%。这说明 SMT 和 ILP [10]（指令级并行，让一个线程内的独立指令同时在流水线里推进）争夺的是同一批空闲发射槽，二者的收益不能简单相乘。

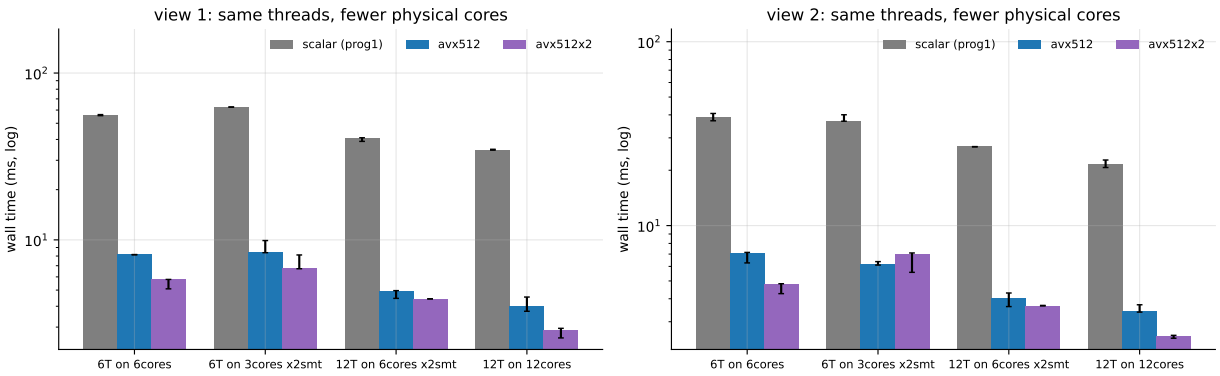



图 5: 相同线程数、不同物理核数的耗时（对数坐标）。灰：标量 prog1 动态调度；蓝：AVX-512 task；紫：AVX-512 双向量交错 task。

**结论：prog1**

Mandelbrot 的工作量集中在图像中部，连续块划分的加速比被最重的块卡住（式 1）；动态划分同时解决了工作量不均和慢核两个问题。这个内核受延迟限制，所以 SMT 几乎能把一个核当两个用。

# 4 Program 2：向量 intrinsics 与 lane 利用率

上一节的问题是“线程之间忙闲不均”。到了 SIMD 内部，同样的问题以另一种形式出现：一条向量指令同时处理 $W$ 个 lane（向量里的一个通道），如果有的 lane 早就做完了，它们只能陪着跑。prog2 就是要把这种浪费量化出来。

官方 prog2 提供了一套“假”向量指令：每个操作都带一个 mask，logger 记录发出的向量指令数和启用的 lane 数。本文按同名 API 重写了 `cs149intrin.h`，`VECTOR_WIDTH` 在编译期指定，分别编译出 W=2/4/8/16 四个可执行文件。

## 4.1 clampedExp

`clampedExpVector` 要对每个元素算一个整数次幂，而每个 lane 的指数不同。做法是用 mask 跟踪哪些 lane 还有剩余指数：每轮循环只对这些 lane 做一次乘法，`_cs149_cntbits(maskActive)` 为 0 时退出；尾部 $N$ mod $W$ 个元素用 `_cs149_init_ones(rem)` 关掉。衡量浪费的指标是 lane 利用率：

$$U = \frac{\sum \text{每条指令启用的 lane 数}}{W \times \text{指令数}}, \tag{2}$$

一组 $W$ 个 lane 的循环次数由其中最大的指数决定，其余 lane 在后面几轮里空转。

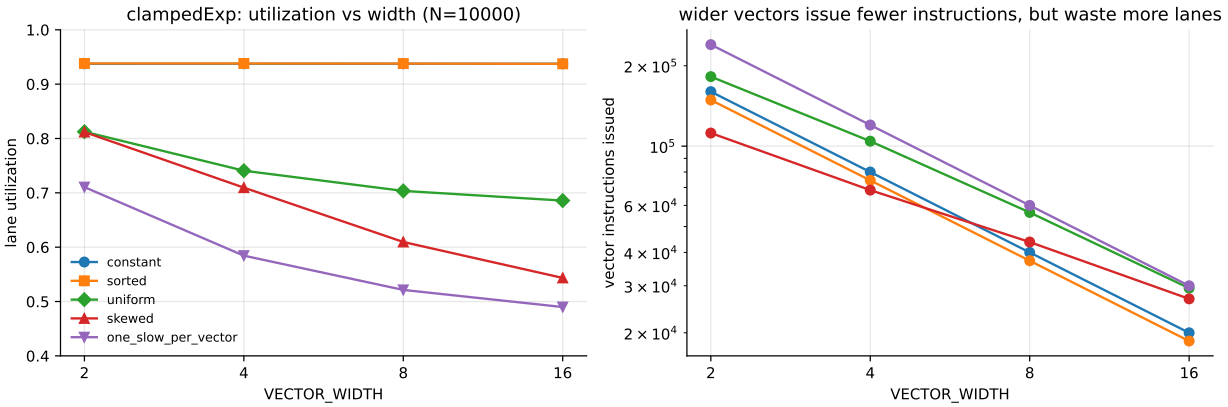



图 6: 左：lane 利用率随向量宽度的变化；右：发出的向量指令数。N=10000。

图 6 和表 3 给出了 5 种指数分布下的结果。

表 3: clampedExp 的 lane 利用率（N=10000，指数取值 0–9）

| 指数分布 | W=2 | W=4 | W=8 | W=16 |
|---|---|---|---|---|
| constant（全为 5） | 0.938 | 0.938 | 0.938 | 0.938 |
| sorted（均匀后排序） | 0.938 | 0.938 | 0.938 | 0.937 |
| uniform（官方设置） | 0.812 | 0.741 | 0.704 | 0.686 |
| skewed（10% 为 9，其余为 1） | 0.812 | 0.710 | 0.610 | 0.543 |
| one_slow_per_vector | 0.711 | 0.584 | 0.521 | 0.490 |

从表里可以读出三件事。

第一，均匀分布下利用率随宽度单调下降。一组里的 lane 越多，组内最大指数就越可能接近 9，其他 lane 空转得也越多。这就是官方问题的答案。

第二，即使指数全部相同（constant），利用率也不是 1。每组 16 条指令里有一条是 `_cs149_vmove_float(result, clampV, maskClamp)`，用来把超过上限的结果截断；而 0.5–1.5 的 5 次方从不超过 9.999999，所以这条指令一个 lane 都不启用，利用率恰好是 $15/16 = 0.9375$。

第三，排序几乎消除了发散。把同样的数据按指数排序后，相邻元素的指数相同，利用率回到 0.938。GPU 上 “按工作量给线程重新分组” 的做法用的是同一个道理，例如光线追踪中的 ray sorting，以及 MoE 中按专家把 token 分组。

### 4.2 arraySum

`arraySum` 是一个没有发散的对照。`arraySumVector` 先用 $N/W$ 次向量加法得到 $W$ 个部分和，再做 $\log_2 W$ 轮 `hadd` + `interleave`，把它们归约到第 0 个 lane。指令数应为 $1+2N/W+2\log_2 W$（一次清零，每个向量一次 load 与一次 add，每轮归约两条）。实测与公式完全一致（W=16、N=10000 时 1259 条），利用率为 1；相对误差 $< 10^{-6}$，来自浮点加法顺序的不同。

## 5 Program 3：SIMD gang 与 task

prog2 用的是模拟的向量指令，prog3 则要在真实硬件上回答：Mandelbrot 用 8 宽的 SIMD，为什么快不到 8 倍？答案有一部分是上一节的发散，但这一节会看到，另一部分来自依赖链的延迟，而且后者还能让加速比反过来超过 SIMD 宽度。

### 5.1 手写 gang：AVX2、AVX-512 与双向量交错

ISPC 把一个函数编译成 $W$ 个 program instance 同时执行（这一组 instance 称为一个 gang），发散的 lane 用 mask 关掉。本文用 intrinsics 直接写出这一过程：每次处理同一行相邻的 8 个（AVX2）或 16 个（AVX-512）像素，维护一个活跃 mask，所有 lane 都逃逸后跳出循环。两种指令集的写法有差别：AVX-512 的比较直接产生 `__mmask16`，计数用 `_mm512_mask_add_epi32`；AVX2 没有 mask 寄存器，需要把比较结果当作 $-1$ 整数相减，并用 `movemask` 判断是否全部结束。下面是 AVX-512 版的内层循环：

Listing 1: AVX-512 gang 的内层循环（`prog3_mandelbrot_simd/main.cpp`）

```
for (int k = 0; k < p.max_iter; ++k) {
   __m512 zr2 = _mm512_mul_ps(zr, zr), zi2 = _mm512_mul_ps(zi, zi);
   active = _mm512_mask_cmp_ps_mask(active, _mm512_add_ps(zr2, zi2), four,
       _CMP_NGT_UQ);
```

```
    if (!active) break; // every lane escaped
    cnt = _mm512_mask_add_epi32(cnt, active, cnt, one);
    __m512 ni = _mm512_mul_ps(_mm512_mul_ps(two, zr), zi);
    zr = _mm512_add_ps(cr, _mm512_sub_ps(zr2, zi2));
    zi = _mm512_add_ps(ci, ni);
}
```

如果只考虑发散，每个 gang 的代价是组内最大的迭代数，模型加速比就是 $W \cdot U_W$，其中

$$U_W = \frac{\sum_i (c_i + 1)}{\sum_{\text{gang}} W \left(\max_{i \in \text{gang}} c_i + 1\right)} \tag{3}$$

可以直接从参考图像算出（view 1：$U_8 = 0.957$、$U_{16} = 0.929$；view 2：$U_8 = 0.831$、$U_{16} = 0.741$）。图 7 把实测值和这个模型放在一起：

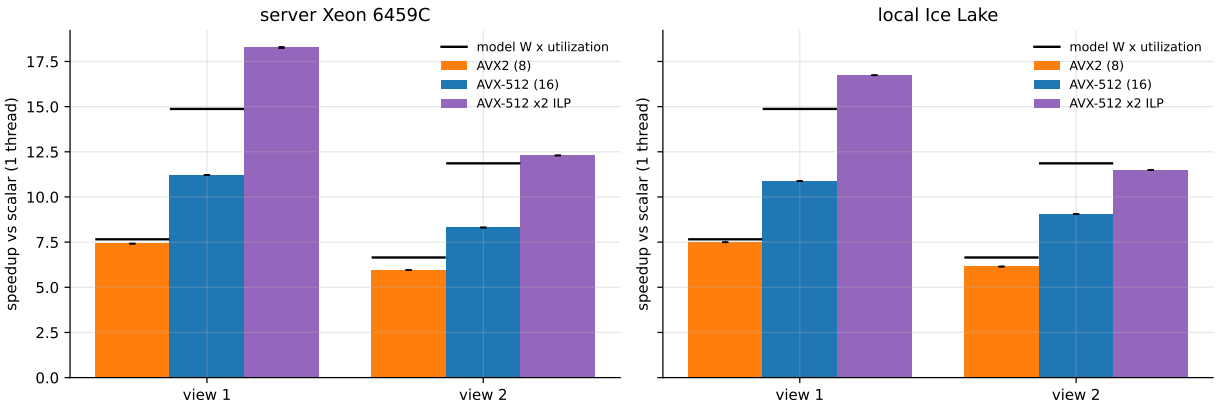



图 7: 单线程 SIMD 加速比（相对标量）。黑线为模型 $W \cdot U_W$；紫色为 AVX-512 双向量交错。

AVX2 实测 7.42 倍，模型 7.66 倍，二者接近，说明 8 lane 版的瓶颈就是发散。view 2 上实测 5.96 倍，模型 6.65 倍。

AVX-512 实测 11.2 倍，比模型的 14.9 倍低得多。每次迭代除了 6 个浮点运算，还有比较、mask 计数和分支，而依赖链的长度与标量完全相同。16 个 lane 只是让每条依赖链变“宽”了，并没有让它变“短”。

双向量交错版（每次推进两组独立的 16 lane，`avx512x2`）达到 18.3 倍，超过了 16 lane 的宽度。两条独立的依赖链交错发射，把上一节 SMT 实验里看到的空闲周期在单线程内填满了。面试里问“为什么 SIMD 加速可以超过 W”，这就是标准例子：标量基线本身没有吃满流水线，所以提高 ILP 的收益会和 SIMD 宽度相乘。

## 5.2 task 数量

单线程的 SIMD 讲完了，接下来把多核加进来。ISPC 的 `launch[n]` 把工作切成 $n$ 个 task 交给线程池，问题是 $n$ 该取多少。图 8 在 12 个工作线程上扫描了 $n$。$n < 12$ 时只有 $n$ 个核在干活。$n = 12$ 和 16 时，每个 task 是连续的一大块行，又回到了连续块划分的不均衡。$n \geq 64$ 后，加速比稳定在约 7.0（AVX-512）到 7.6（双向量交错）倍（相对同一 ISA 的单线程）。所以 task 数应当远多于核数，让线程池在运行时做负载均衡。官方答案通常给出“几十到几百个 task”，原因就在这里。

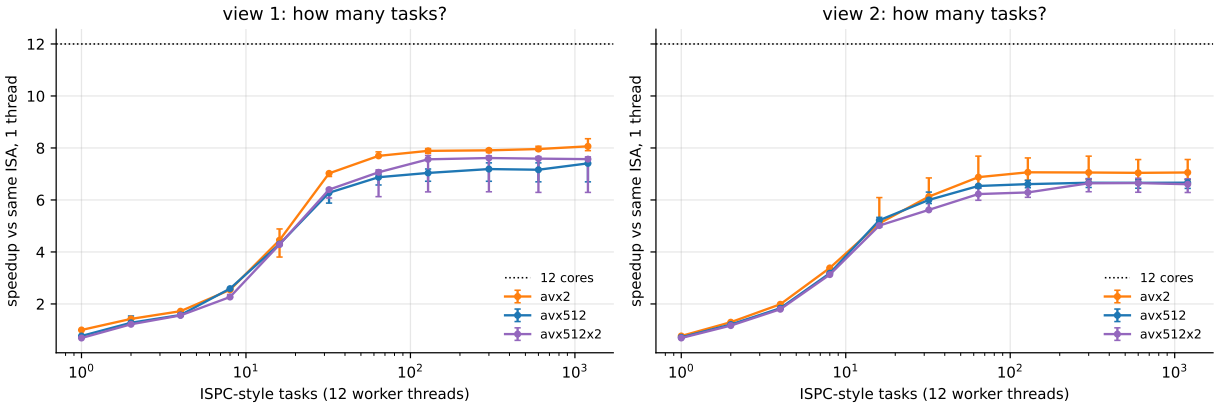



图 8: ISPC 式 task 数量扫描（12 个工作线程）：相对同一 ISA 单线程的加速比。

## 5.3 把 SIMD、ILP、多核和 GPU 乘起来

到这里，单核内的 SIMD 与 ILP、多核之间的 task 都测过了。它们叠在一起能走多远？图 9 把 view 1 和 view 2 上每一步的加速画成一级级“阶梯”，依次是标量、AVX2、AVX-512、双向量交错、12 核，最后是 RTX 5090。CPU 上各级加速基本相乘：SIMD 约 11 倍、ILP 约 1.6 倍、12 核约 7.6 倍，合计约 139 倍（view 1）与 82 倍（view 2）。RTX 5090 在同样 1600 × 1200 的图像上达到 8233 倍与 5762 倍；把图像放大 16 倍以摊薄 launch 开销后，为 10645 倍与 8267 倍。也就是说，GPU 比最好的 12 核 CPU 版本还快 60–100 倍，第 9 节会解释这个差距从哪里来。

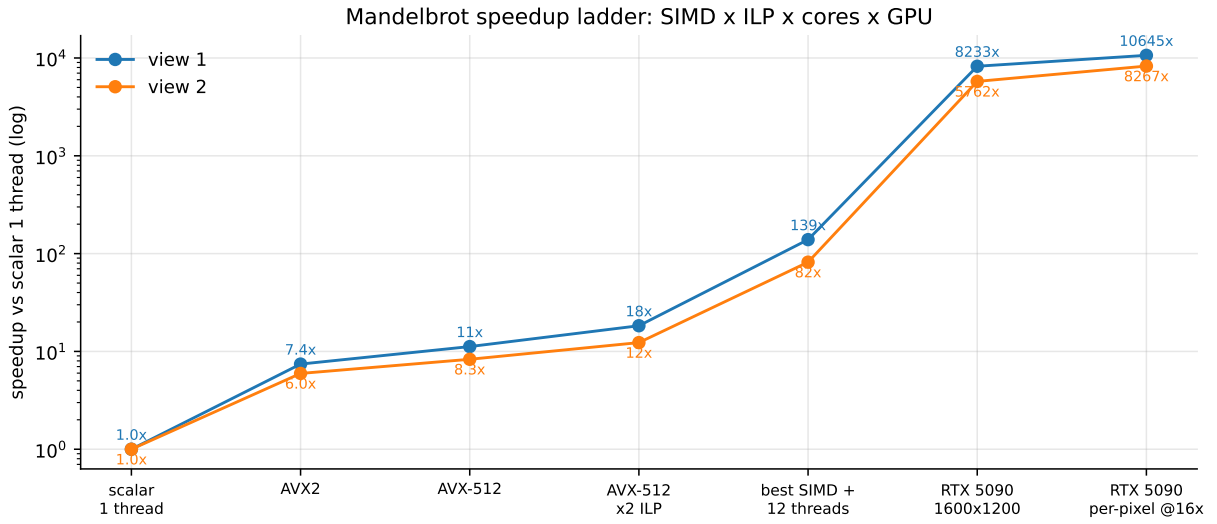



图 9: Mandelbrot 加速阶梯（相对标量单线程，对数坐标）。

## 5.4 一个测量陷阱：worker 放在哪个 CPU 上

扫描 task 时我发现了一个反常：“1 个 task、1 个工作线程”比在主线程上直接调用同一个函数指针慢约 23%（相对速度 0.77）。同样的代码、同样的工作量，只是换了个线程跑，不应该有区别。

我最初怀疑工作线程绑在了承担中断的 CPU 0 上，于是对比了三种情况：绑在 CPU 0、绑在另一个核、不绑核（图 10）。结果三者在 3 轮中的 8 次里都是 0.77 左右，只有 1 次接近 1.0，这个假设被否定了。剩下的区别是：工作线程每次计时都是新建的，而主线程刚刚连续跑过多次同样的内核。这与上一节粒度扫描中观察到的“冷核”现象一致。

再看绝对值，能进一步缩小范围。view 1 上工作线程比主线程多 8.9 ms（38.6 对 29.7 ms），view 2 上同样多 8.9 ms（31.3 对 22.4 ms）。两个视图的工作量差 1.3 倍，多出来的时间却相同，所以这是每次 launch 都要付的一笔加性的固定代价，而不是工作线程整体跑得慢。

本机 4 核上同样的实验给出了不同的图景：绑在 CPU 0 时相对速度为 0.75 / 0.90，绑在其他核或不绑核时为 0.92–1.00，即本机只有 CPU 0 慢（推测与中断集中投递到 CPU 0 有关，未验证）。A2 报告用 CPU 时间进一步区分了 “等待” 和 “执行变慢”，结论是服务器上的 worker 在没有线程自旋时每条指令更贵，最可能的原因是宿主机的频率与空闲管理；但没有 `turbostat`，无法最终确认。

这个陷阱的影响是：图 8 中 “相对单线程 SIMD” 的加速比被系统性地低估了约 20%。“相对标量” 的数字（图 9）不受影响，因为标量基线也是在主线程上测的，阶梯图只比较墙钟时间。更稳妥的做法是像 prog5 那样使用常驻线程池；本文在 prog3 中保留了每次新建线程的实现，是为了与官方 ISPC task 的语义对应。

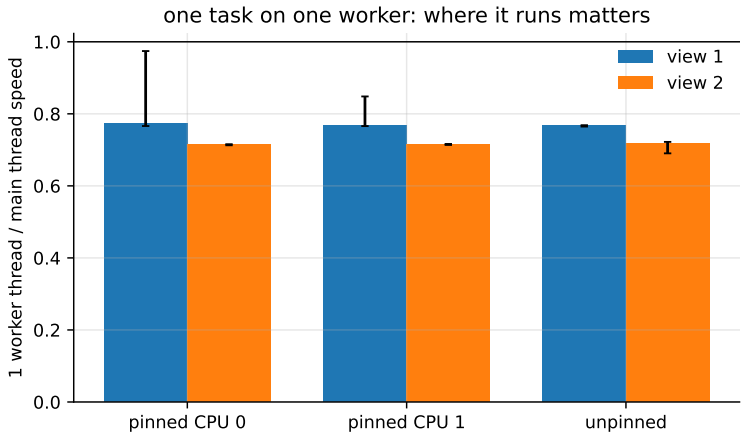



图 10: 1 个 task 在 1 个工作线程上运行时相对主线程的速度，按绑核位置区分。

## 6 Program 4：Newton sqrt 与最好、最坏输入

前面几节说明了发散会让 SIMD 打折扣。prog4 把这件事推到极端：能不能构造一组输入，让 SIMD 版本比标量还慢？

prog4 用 Newton 法求 $1/\sqrt{x}$，初值为 1，误差阈值为 $10^{-5}$。迭代次数完全由输入决定（图 11 左）：$x = 1$ 时 0 次，$x \to 3$ 时约 20 次，$x \to 0$ 时更多。据此可以构造出两个极端：

- 最好的输入（all_slow，全部为 2.998）：每个元素都要迭代 20 次，没有发散，而且计算量大到足以摊薄 load 和 store 的开销。AVX-512 达到 13.5 倍，再乘上 12 线程达到 104 倍。
- 最坏的输入（one_slow_W，每 W 个元素中 1 个为 2.998，其余为 1.0）：每个向量都必须跑满 20 次，但有效工作只有 $1/W$。AVX2 在 one_slow_8 上只有 0.72 倍，比标量还慢，这正是官方要求构造的情形。

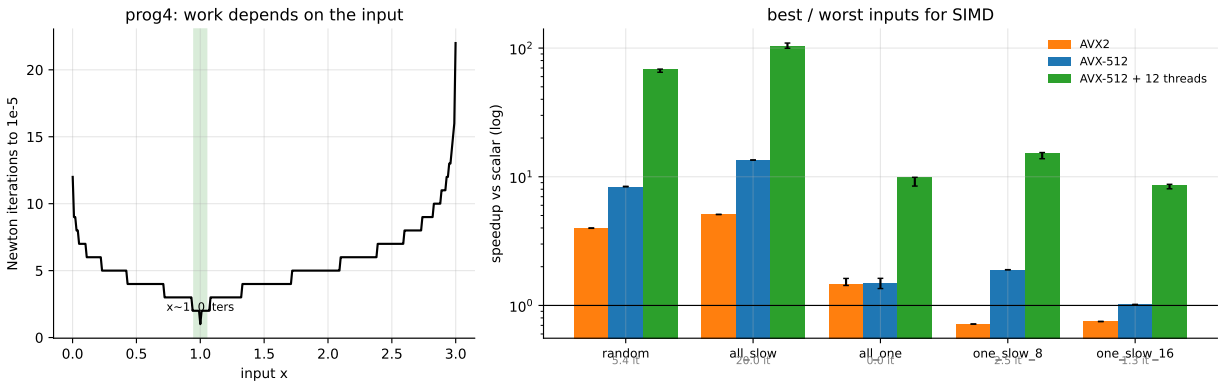



图 11: 左：迭代次数随输入的变化；右：各输入模式的加速比（对数坐标），灰字为平均迭代次数。

表 4 列出了耗时。最能说明发散代价的是 AVX-512 一列：all_slow、one_slow_8、one_slow_16 三种输入的平均迭代次数分别是 20、2.5、1.25，耗时却是 96.8、96.8、97.3 ms，几乎完全相同。AVX2 在 all_slow 与 one_slow_8 上也同为 256 ms。对 SIMD 而言，一个向量的代价只取决于其中最慢的那个 lane，其余 lane 做多做少都不影响。

表 4: prog4 耗时（N = 2000 万，服务器 3 轮中位数）

| 输入 | 平均迭代 | 标量 ms | AVX2 ms | AVX-512 ms | AVX-512 加速 | +12 线程 |
|---|---|---|---|---|---|---|
| random | 5.35 | 742.7 | 186.8 | 88.9 | 8.36 | 68.5 |
| all_slow | 20.0 | 1305.5 | 256.2 | 96.8 | 13.48 | 104.1 |
| all_one | 0.0 | 13.3 | 9.1 | 9.0 | 1.48 | 9.9 |
| one_slow_8 | 2.50 | 183.3 | 256.1 | 96.8 | 1.89 | 15.2 |
| one_slow_16 | 1.25 | 99.0 | 131.7 | 97.3 | 1.02 | 8.6 |

表里还有两个细节值得追问。

为什么 AVX-512 比 AVX2 快 2.6 倍，而不是宽度之比的 2 倍？因为 AVX2 没有 mask 寄存器：冻结已经收敛的 lane 要用 `blendv`（2 个 uop），判断是否全部收敛还要一条 `movemask`。AVX-512 的比较直接写 mask，更新用 `mask_mov`，每次迭代的指令更少。

为什么 AVX2 在 all_slow 上只有 5.1 倍？标量 all_slow 每个元素每次迭代平均耗时 $1305.5\,\text{ms}/(2\times10^7\times20)=3.3$ ns，约 11 个周期；而一次迭代的依赖链（3 次乘、1 次减、1 次乘）约需 20 个周期。实际比依赖链还快，说明乱序执行在预测正确的 20 次循环之后，提前开始了下一个元素，让两个元素的依赖链重叠起来。换句话说，标量基线自带了约 2 倍的 ILP，SIMD 版本里却没有这部分，所以加速比打了折扣。

## 7 Program 5：saxpy 与内存带宽

前四个 program 的瓶颈都在“算”上。saxpy 则相反：它几乎不算，只搬数据。这一节要回答的是，这类程序加核、加 SIMD 宽度还有没有用。

saxpy 计算 $r=ax+y$，每个元素做 2 次浮点运算、读 8 字节、写 4 字节，所以算术强度（每读写一个字节做多少次浮点运算）为 $I=2/12\approx0.17$ FLOP/B。roofline [3] 模型把可达性能写成

$$P=\min(P_{\text{peak}},\ I\cdot B) \tag{4}$$

其中 $B$ 是内存带宽。$I$ 远小于任何 CPU 的拐点（约 10–50 FLOP/B），所以 saxpy 的性能完全由带宽 $B$ 决定。

## 7.1 带宽随工作集的变化

带宽并不是一个固定值：数据落在哪一级缓存，带宽就差好几倍。图 12 扫描了工作集（三个数组的总大小）。单线程时，工作集小于 L1（48 KB）的 SIMD 内核达到约 229 GB/s，进入 L2 后约 125 GB/s；超过 L2（2 MB）后跌到 35 GB/s，超过 L3 后只剩约 18 GB/s。12 线程时私有 L2 的总容量相当于 $12 \times 2$ MB，工作集 12.6 MB 时总带宽超过 1 TB/s；超过 L3 后降到 130 GB/s 左右。

曲线的左端也值得一看：12 线程在这里反而很低。原因是每次调用只有几微秒，线程池同步的开销占了主导。小问题不值得并行。

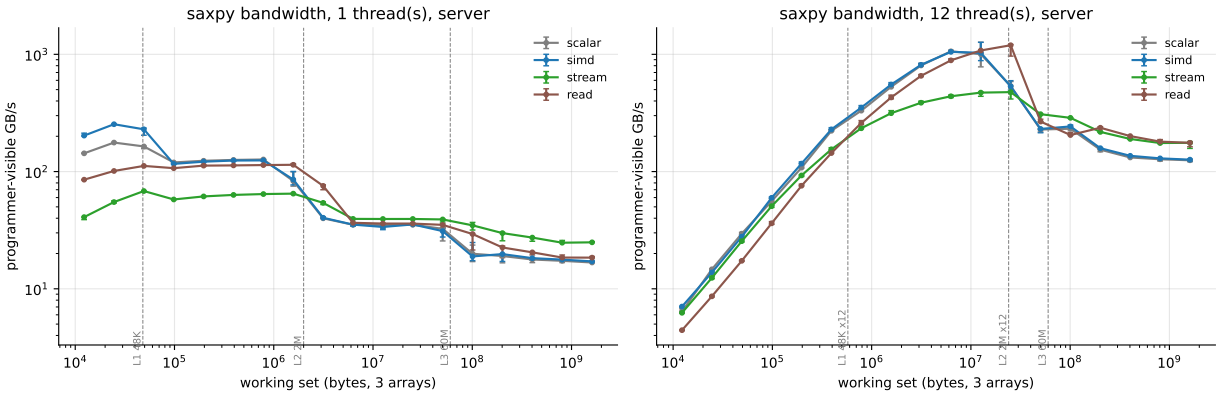



图 12: saxpy 程序员可见带宽（12 B/元素）随工作集变化，虚线为各级缓存容量（多线程时按线程数放大私有缓存）。

## 7.2 write-allocate 与 streaming store

程序员以为 saxpy 每个元素搬 12 字节，但 DRAM 实际看到的要多。普通 store 写入一个不在缓存中的行时，CPU 会先把整行读进来（read-for-ownership，即为了获得写权限而先读），所以 saxpy 的真实 DRAM 流量是 16 B/元素。`_mm512_stream_ps` 是 non-temporal store，绕过缓存直接写内存，省掉了这次读，流量回到 12 B/元素。

DRAM 区间的结果符合这笔账。单线程 stream 的可见带宽是 25 GB/s，普通 SIMD 是 18 GB/s，比值 1.39；12 线程为 175 对 130 GB/s，比值 1.35。两个比值都接近 $16/12 = 1.33$。但在缓存区间，stream 很差（单线程 L1 区间 68 对 229 GB/s），因为数据本可以留在缓存里，non-temporal store 却强行把它写回内存。所以 streaming store 只适合 “写完很久都不会再读” 的大数组。

## 7.3 线程扩展与 NUMA

既然 saxpy 受带宽限制，加核还能不能提速？图 13 左图是 768 MB 工作集的真实 DRAM 带宽 [7]（含 write-allocate [8]）随线程数的变化：单线程 23 GB/s，12 线程 173 GB/s，几乎线性增长，还没有饱和。这与官方作业在四核桌面机上看到的 “两三个线程就饱和” 不同。Sapphire Rapids 每路有 8 个 DDR5 通道，单核的带宽受限于它自己能同时挂起多少个 cache miss，而不是内存控制器。按 Little [11] 定律，带宽 = 并发请求数 $\times$ 64 B / 延迟，所以每多一个核，就多一份并发请求。本机四核虚拟机同样没有饱和（4 线程约 59 GB/s）。

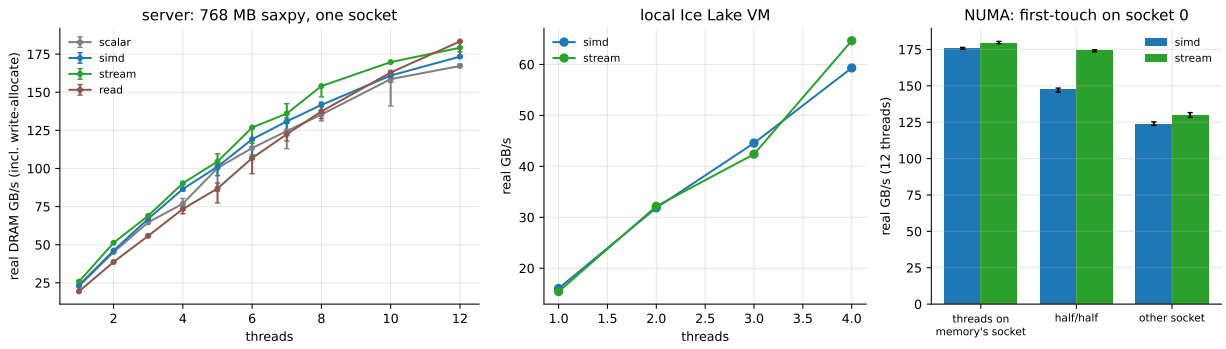



图 13: 左：服务器 DRAM 带宽随线程数变化；中：本机；右：NUMA 放置（数据由主线程 first-touch 在 socket 0）。

在双路服务器上，还要关心数据放在哪个 socket 的内存上。右图是 NUMA 实验：数组由主线程初始化，按 first-touch 策略（页面落在第一次写它的线程所在节点）全部落在 socket 0 的内存上。12 个线程都在 socket 0 时，SIMD 内核为 175 GB/s；全部在 socket 1 时为 123 GB/s，下降 30%；两边各一半时为 147 GB/s。因此在双路服务器上做内存密集的计算，数据初始化也要并行化，并让每个线程初始化自己将要处理的那一段。

# 8 Program 6：K-means 的 profiling 与优化

前面五个 program 都是一个内核、一种瓶颈。真实程序往往由几个阶段组成，只优化最显眼的那个阶段，收益会被其余部分吃掉。prog6 用 K-means 演示这一点。

官方 prog6 给出一个慢的 K-means（M = 100 万个点、N = 100 维、K = 3），要求先计时找瓶颈，再优化。本文的 baseline 照搬官方结构：`computeAssignments` 外层循环 K、内层循环 M，计算带 `sqrt` 的欧氏距离；`computeCentroids` 和 `computeCost` 串行执行。数据由 8 个高斯簇生成，收敛需要 3 轮迭代。图 14 是各阶段的耗时：

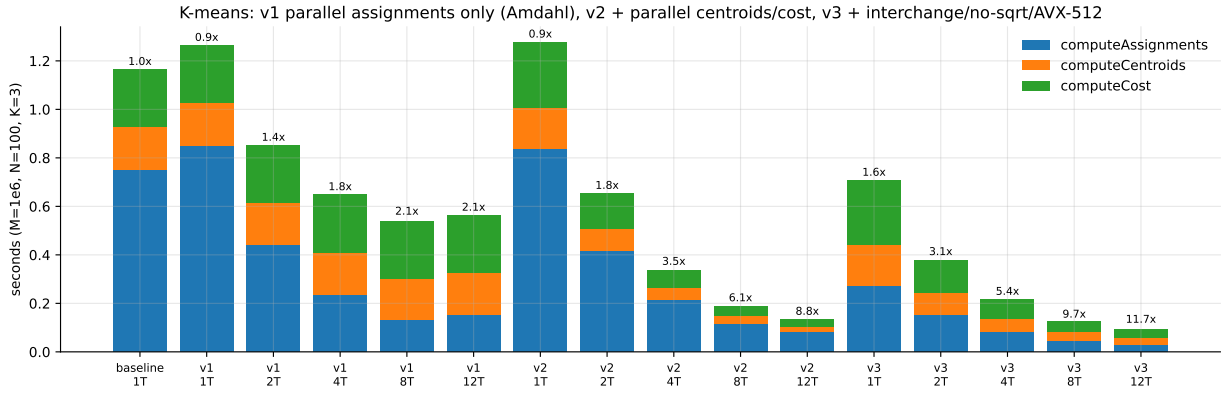



图 14: K-means 分阶段耗时与加速比（服务器 3 轮中位数）。

baseline 总计 1.163 s，其中 `computeAssignments` 占 0.752 s（65%），`computeCentroids` 0.176 s，`computeCost` 0.238 s。

v1 是官方期望的改法：按点 $m$ 把 `computeAssignments` 分给 $T$ 个线程。8 线程时这个阶段降到 0.131 s，但另两个阶段仍要 0.41 s，总加速只有 2.1 倍。Amdahl [4] 定律给出的上限是 $1/(1-0.65) = 2.8$ 倍，再多的线程也越不过去。

v2 把 `computeCentroids` 和 `computeCost` 也按点并行。每个线程累加到私有的部分和，最后再合并，这样避免了对共享数组的原子操作和 false sharing（不同线程写同一缓存行上的不同变量，导致缓存行在核之间来回迁移）。12 线程时达到 8.8 倍。

v3 在此基础上做了三处单线程优化：(a) 循环交换为外层 $m$、内层 $k$，每个点只从内存读一次，而 baseline 每个点要读 K 次，共 2.4 GB；(b) 比较平方距离，省掉 `sqrt`，因为单调函数不改变 argmin；(c) 用 AVX-512 计算距离。单线程即达 1.64 倍，12 线程 11.7 倍。

所有版本的聚类分配都与 baseline 完全一致（mismatch = 0），迭代次数也相同。v3 在 8 线程以后，`computeAssignments` 每轮约 15 ms，相当于每秒读约 55 GB 数据，已经接近这一阶段的带宽与线程启动开销共同决定的上限。

# 9 GPU 对照：同样的三个问题在 SIMT 上

CPU 上的三个结论（负载要均衡、一个向量的代价由最慢的 lane 决定、低算术强度的算子只看带宽）换到 GPU 上还成立吗？GPU 的 warp 就是一个 32 lane 的 gang，只是 mask 由硬件维护。所以我把 prog3、prog4、prog5、prog6 的内核写成 CUDA（`gpu/a1_gpu.cu`），逐一检验哪些结论能搬过去、哪些不能。

## 9.1 Mandelbrot：warp 效率与 occupancy

一个线程计算一个像素，warp 由 block 内连续的 32 个线程组成。于是 block 的形状决定了一个 warp 覆盖的是一行 32 个像素，还是一个 $8 \times 4$ 的小方块。后者的像素在空间上更集中，迭代数更接近，warp 执行效率更高。图 15 在 $6400 \times 4800$ 的图像上扫描了 12 种形状：

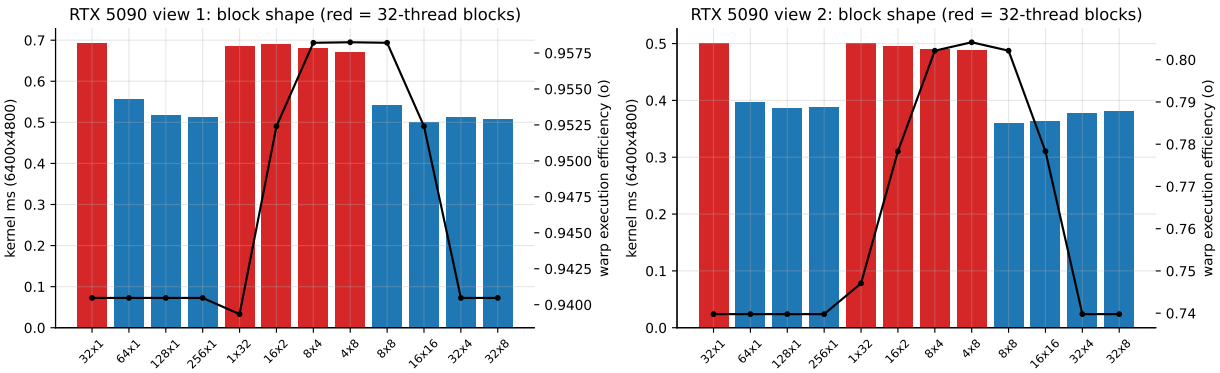



图 15: RTX 5090 上 block 形状对 kernel 时间（柱）与 warp 执行效率（点线）的影响；红色柱为每个 block 只有 32 个线程的形状。

2D 方块的 warp 效率确实更高（view 1：$8 \times 4$ 为 0.958，$32 \times 1$ 为 0.940；view 2：0.802 对 0.740），这与 prog2 中 “排序减少发散” 是同一个道理。

但每个 block 只有 32 个线程的形状（红色柱）都慢。原因是 occupancy（每个 SM 上同时驻留的线程数）：每个 SM 能驻留的 block 数有上限（sm_120 为 32 个），32 线程的 block 让每个 SM 最多只有 1024 个线程，用来隐藏延迟的 warp 不够。$8 \times 8$ 与 $16 \times 16$ 兼顾了空间局部性和 occupancy，是最快的两种（view 2 为 0.360 ms 与 0.364 ms，$32 \times 1$ 为 0.501 ms）。

## 9.2 sqrt：GPU 上发散为什么 “不见了”

在 CPU 上，sqrt 的发散代价非常明显。同一个 Newton sqrt 在 RTX 5090 上处理 2000 万个元素时，所有输入模式的耗时却都在 0.096–0.108 ms 之间（图 16 右）。读写 160 MB 用 0.104 ms，相当于

1.54 TB/s，说明 kernel 被显存带宽限制住了，计算（即使是发散的计算）被完全隐藏在访存后面。

为了让发散重新露面，我把每个元素的求解重复 $R$ 次（初值略作扰动，避免被编译器消除），让 kernel 逐渐变成计算受限（图 16 左）。$R = 64$ 时：

- one_slow_8、one_slow_16、one_slow_32 的耗时（5.05、5.04、5.03 ms）与 all_slow（5.03 ms）完全相同。warp 宽 32，只要每 32 个元素里有一个慢元素，整个 warp 都要跑满 20 次迭代。这与 CPU 上 AVX-512 在 one_slow_16 上的结论一致，只是 GPU 的 “gang” 更宽。
- one_slow_64 为 2.81 ms，约为 all_slow 的一半，因为只有一半的 warp 含有慢元素。
- random 为 3.35 ms，是 all_slow 的 67%，而它的平均迭代数只有 all_slow 的 27%（5.35 对 20）。把同样的数据排序后（sorted_random）降到 1.59 ms，快 2.1 倍。这就是 GPU 上 “按工作量重排线程” 的收益，对应 prog2 中排序把 lane 利用率从 0.69 提到 0.94。

由此可见，“发散有多贵” 并不是一个常数，它取决于内核在 roofline 上的位置：同一个发散的内核，带宽受限时代价为零，计算受限时代价与 CPU 的 SIMD 完全相同。

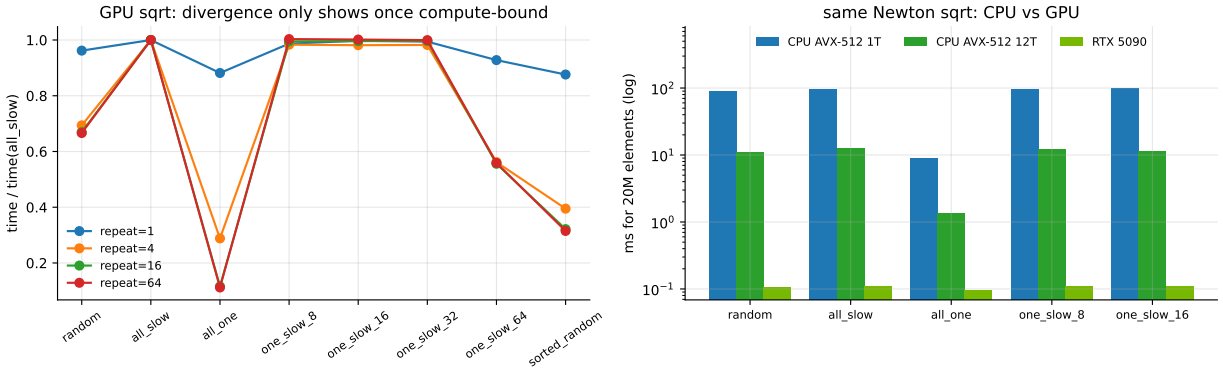



图 16: 左：各输入相对 all_slow 的耗时，$R$ 为每个元素重复求解的次数；右：同一 sqrt 在 CPU 与 GPU 上的耗时（2000 万元素）。

## 9.3 saxpy：GPU 赢在显存，输在 PCIe

saxpy 只看带宽，而 GPU 的显存带宽比 CPU 内存高一个数量级，看上去 GPU 稳赢。图 17 把 GPU saxpy 与 CPU 放在同一个坐标系里：显存区间 GPU 稳定在约 1.55 TB/s（标称 1.79 TB/s 的 87%），L2（96 MB）区间峰值约 6.1 TB/s。可一旦算上把输入从 host 拷到 device、再把结果拷回来，端到端只剩约 52 GB/s，比服务器 CPU 12 线程的 130 GB/s 还低。这正是 CS149 Assignment 3 Part 1 要学生得出的结论：带宽受限的小算子不值得为它把数据搬上 GPU。GPU 的优势要靠让数据常驻显存、把多个算子融合在一次搬运里才能兑现。

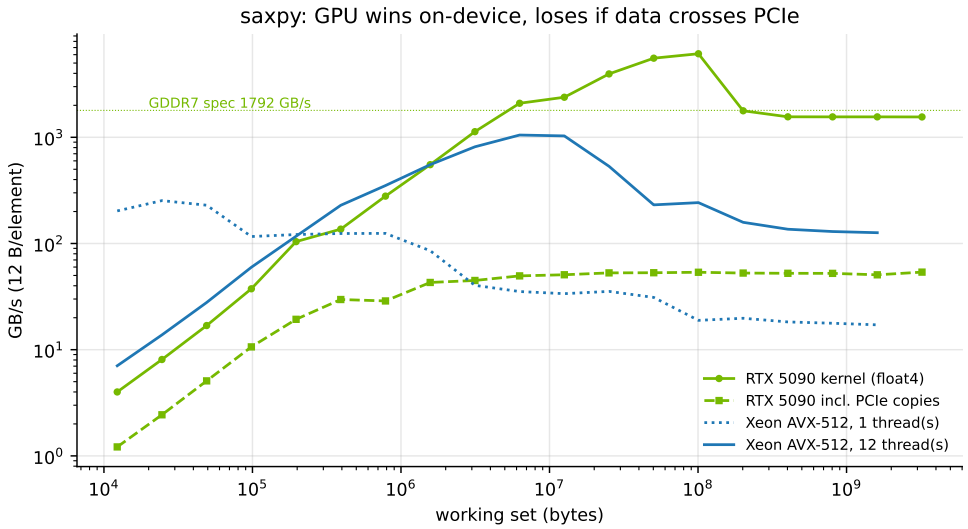



图 17: saxpy 带宽：RTX 5090（kernel、含 PCIe 拷贝）与 Xeon AVX-512（1 线程、12 线程）。

## 9.4 K-means 的 assignment kernel：合并访存

K-means 在 GPU 上又多了一个 CPU 上不明显的问题：数据布局。每个线程负责一个点。数据按点存储（AoS，$M \times N$）时，同一 warp 的 32 个线程在同一时刻读取相距 800 B 的地址，硬件无法把它们合并成少数几次访存；转置成按维存储（SoA，$N \times M$）后，相邻线程读相邻地址，访存可以合并。实测 AoS 为 3.74 ms（214 GB/s 有效带宽），SoA 为 1.43 ms（560 GB/s），快 2.6 倍，结果与 CPU 参考逐个一致。与 CPU v3 的 12 线程（每轮约 11 ms）相比，GPU SoA 版本快约 8 倍。

## 9.5 为什么 GPU 比 12 核 AVX-512 快 60–100 倍

回到第 5 节留下的问题。先用 “每秒能做多少次 lane 运算” 粗算峰值：RTX 5090 有 170 个 SM，每个 SM 128 个 FP32 lane，频率约 2.4 GHz，约 $5.2 \times 10^{13}$ lane-op/s；12 个 Xeon 核每核 2 个 512 bit 浮点端口、16 lane、约 3 GHz，约 $1.2 \times 10^{12}$ lane-op/s。峰值相差约 45 倍。实测 GPU 每秒完成约 $4.5 \times 10^{12}$ 次有效的像素迭代（图 18），每次迭代约 8 个运算，即约 $3.6 \times 10^{13}$ op/s，达到峰值的约 70%；CPU 最好的版本比 GPU 慢 59 倍，折算约为自身峰值的 50%。

剩下的差距来自延迟隐藏。GPU 每个 SM 同时驻留几十个 warp，一个 warp 等乘法结果时，调度器就发射另一个 warp，所以依赖链延迟几乎被完全隐藏。CPU 每个核只有 2 个硬件线程，依赖链延迟只能靠 SMT 和手工交错（第 5 节，约 1.6 倍）部分弥补。图 18 还显示，迭代上限只有 64 时 GPU 吞吐明显偏低：kernel 只运行约 0.17 ms，launch 与最后一波（tail wave）block 的开销占比很大。

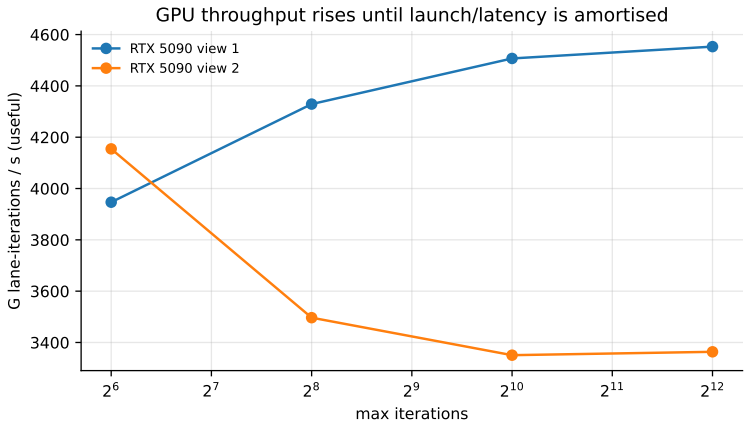
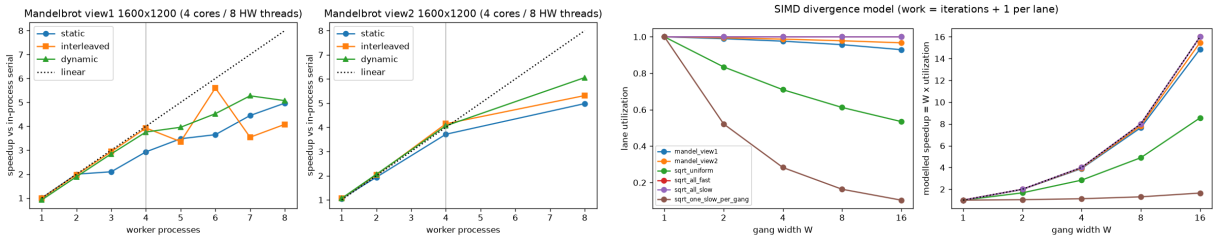



图 18: RTX 5090 Mandelbrot（6400 × 4800）有效迭代吞吐随最大迭代数的变化（取 12 种 block 形状中的最好值）。

# 10 仓库原有 Python 版本的回顾

仓库最初用 Python/numpy 实现了 prog1、prog2、prog4、prog5 的核心问题：用进程池代替线程，用 gang 利用率模型代替真实 SIMD，用 numpy 掩码模拟发散执行。图 19 是其中两张图。它们的定性结论与 C++ 版一致（连续块不均衡、发散有代价、带宽很快饱和），但数字不能用来回答作业。原因有三个：numpy 每次调用都有固定开销；Python 版 Mandelbrot 的单像素代价与块大小有关（详见实验 README 中的“测量陷阱”）；进程间通信又引入了与并行无关的开销。C++ 版的单线程 Mandelbrot（0.33 s）比 numpy 版（1.28 s）快约 4 倍，SIMD 版快约 45 倍，后者才是作业想要测量的东西。




(a) Python 进程池版 prog1　　(b) gang 利用率模型

图 19: 仓库原有 Python 版本的结果（定性参考）。

# 11 面试专题

## 11.1 20 秒版本

并行加速有三个杀手：负载不均、SIMD 发散和带宽。Mandelbrot 中间贵，所以连续块划分在 12 核上只有 5.89 倍，换成动态分块是 11.1 倍。一个向量的代价由最慢的 lane 决定，sqrt 每 8 个元素里混一个慢的，AVX2 就比标量还慢。saxpy 每 12 字节只做 2 次浮点运算，单核只能拿到 23 GB/s，加到 12 个核也只有 173 GB/s；搬到 GPU 上如果要过 PCIe，只剩 52 GB/s。另外，这个 Mandelbrot 内核受延迟限制，所以 SMT 和 ILP 都能带来接近翻倍的收益。

## 11.2 追问与答案

**面试追问：为什么 3 个线程比 2 个线程还慢？**

**答：**因为连续块划分时，墙钟时间由最重的那一块决定。view 1 三等分时，中间那块包含了整个 Mandelbrot 集的主体，工作量是平均的 1.8 倍以上，所以加速比只有约 1.6 倍；2 线程时两块对称，各约一半，反而更好。解决办法是隔行或动态划分。

**常见错答：**归因于 “线程创建开销” 或 “缓存冲突”。线程只创建了 3 个，开销是微秒级；每线程计时图直接显示问题出在工作量不均。

**面试追问：超线程对 Mandelbrot 有用吗？**

**答：**有用，而且接近翻倍。实测 6 个线程放在 3 个物理核上，耗时只比放在 6 个独立核上多约 11%。原因是内层循环是一条乘加依赖链，单线程每个周期只能发射很少的指令，另一个硬件线程的独立依赖链正好填满空闲端口。反过来，如果代码已经吃满执行端口（例如高度优化的 GEMM），SMT 的收益会接近 0。本文的双向量交错版就是证据：它在单线程内已经交错了两条依赖链，从 SMT 得到的收益明显少于单向量版。

**常见错答：**“超线程只有 20%–30% 收益”，这只是平均值，不是规律；或者 “超线程共享执行单元，所以没用”。

**面试追问：8 宽 SIMD 为什么 Mandelbrot 只有约 7.4 倍、而 16 宽只有约 11 倍？**

**答：**8 宽主要输给发散，16 宽还输给延迟。模型加速比是 $W \times U_W$，view 1 的 $U_8 = 0.957$、$U_{16} = 0.929$，理论值是 7.66 和 14.9。AVX2 接近模型；AVX-512 还受依赖链延迟限制，每次迭代的比较、mask 计数和分支也要占发射槽。在单线程里交错两组向量后，AVX-512 达到约 18.3 倍，超过 16。

**常见错答：**“AVX-512 降频”。Sapphire Rapids 上降频很小，而双向量交错版同样用 512 bit 却快 1.6 倍，说明主因是延迟而不是频率。

**面试追问：怎样构造让 SIMD 比标量还慢的输入？**

**答：**让每个向量里恰好有一个代价最高的 lane，其余 lane 代价接近 0。sqrt 中每 8 个元素放一个 2.998、其余放 1.0，每个向量都要跑 20 次迭代，有效工作却只有 1/8；再加上 SIMD 版本每次迭代额外的 blend 和 movemask 指令，AVX2 只有 0.72 倍。

**常见错答：**“让全部输入都很慢”。那反而是最好的情况（13.5 倍），因为没有发散。

**面试追问：saxpy 为什么加核加不上去？write-allocate 是什么？**

**答：**saxpy 的算术强度只有 0.17 FLOP/B，完全受带宽限制。单核的带宽受限于自身能挂起的 cache miss 数（Little 定律），多核把这些并发请求累加起来，直到内存控制器饱和。write-allocate 是指写一个不在缓存中的行时，CPU 要先读后写，所以真实流量是 16 B/元素；non-temporal store 省掉这次读，DRAM 区间带宽提高约 1/3，但在缓存区间反而变慢。

**常见错答：**“用 AVX-512 就能更快”。计算单元早已空闲，SIMD 宽度和带宽无关。

**面试追问：K-means 只把 computeAssignments 并行化，8 核为什么只有 2 倍？**

**答：**这是 Amdahl 定律。该阶段占 65%，其余 35% 仍是串行的，上限是 $1/0.35 = 2.8$ 倍，实测 2.1 倍。要继续提速，就必须并行另外两个阶段（用线程私有的部分和避免原子操作），并减少每个阶段的工作量：循环交换让每个点只读一次，比较平方距离省掉 sqrt。

**常见错答：**只回答 “用更多线程”；或者把 centroids 的累加写成对共享数组的 atomicAdd，导致严重的争用与 false sharing。

**面试追问：task 数量应该设成多少？**

**答：**远多于核数。task 数等于核数时，每个 task 是一大块连续行，又回到了连续块划分的不均衡；64 个以上的 task 让线程池在运行时做均衡，加速比就到达平台。上限由每个 task 的调度开销决定，一般让每个 task 至少运行几十微秒。

**常见错答：**“等于核数最好，可以避免上下文切换”。task 不是线程，task 多不会导致线程切换。

**面试追问：GPU 上的 warp 发散和 CPU 的 SIMD 发散有什么不同？**

**答：**机制相同，代价不同。两者都是一组 lane 共享指令流，用 mask 关掉不活跃的 lane；但 GPU 的内核常常受显存带宽限制，发散的计算会被访存隐藏。本文的 sqrt 在 RTX 5090 上所有输入耗时相同（约 0.1 ms，1.54 TB/s），用重复计算把它变成计算受限之后，发散的代价才显现出来。另外，GPU 靠 occupancy 隐藏延迟，所以选 block 形状时要同时考虑 warp 内的空间相关性和每个 SM 驻留的线程数。

**常见错答：**“GPU 不怕发散”，或者 “GPU 发散一定是 32 倍的代价”。

**面试追问：什么时候不该把计算搬到 GPU？**

**答：**当数据要过 PCIe、而计算量相对数据量很小时。saxpy 在 GPU 上的 kernel 带宽是 1.55 TB/s，加上 H2D/D2H 拷贝后只有 52 GB/s，比服务器 CPU 的 130 GB/s 还低。算术强度低的算子要么让数据常驻显存，要么与前后算子融合。

**常见错答：**只比较 kernel 时间。

### 11.3 STAR 故事

**情境：**在一台多租户的双路 Xeon 服务器上复现 CS149 Assignment 1，并要求结果可重复。**任务：**测出负载均衡、SIMD 发散、带宽三类瓶颈，给出逐位正确的 SIMD、多线程和 GPU 实现。**行动：**先采样每个 CPU 的忙闲，避开被其他租户占满的核，并固定绑核；用 `-ffp-contract=off` 和与标量一致的运算顺序保证 SIMD 结果逐位相等；实验重复 3 轮，报告中位数与区间。发现 SMT 接近翻倍之后，我推断内核受延迟限制，于是写出双向量交错版本来验证。**结果：**Mandelbrot 单线程 AVX-512 从 11.2 倍提到约 18.3 倍，12 核合计约 139 倍（view 1）与 82 倍（view 2）；K-means 从 2.1 倍（只并行一个阶段）提到 11.7 倍；并定量说明了 saxpy 搬到 GPU 后端到端反而变慢的原因。

## 12 局限

**不能推出的结论**

- 服务器是多租户宿主机，cgroup 配额为 16 个 CPU，其他租户的负载随时间变化。3 轮实验的区间已在图中给出，但不能排除系统性偏差。
- 本文没有 ISPC 编译器，gang 由手写 intrinsics 实现；ISPC 生成的代码在 mask 处理和指令调度上可能更好，也可能更差。
- 服务器单线程 turbo 频率高于全核频率，多线程加速比因此偏低；本文没有锁定频率。
- 本机是虚拟机，SMT 兄弟关系来自虚拟化层报告的拓扑，不一定对应物理核。
- GPU 结论来自单张 RTX 5090。消费级卡的 FP64 很弱，K-means 的 double 计算在数据中心卡上会快得多。
- 所有 GPU 数字都是 kernel 时间（CUDA event），saxpy 端到端另行测量；没有使用 CUDA Graph 等手段降低 launch 开销。

# 13 结论

本文用 C++ 与 CUDA 完成了 CS149 Assignment 1 的全部六个 program，并用同一套方法在服务器、本机四核和 RTX 5090 三个平台上做了测量。读完这份报告，最值得带走的是下面几点。

负载均衡要看最重的那一份，而不是平均值；动态调度 [5] 同时解决了工作量不均和慢核两个问题。SIMD 的代价由最慢的 lane 决定，但发散到底有多贵，要看内核是计算受限还是带宽受限。带宽受限的内核，加核只是把每个核的并发请求线性地累加起来；streaming store 与 NUMA 放置各自能影响约 30%，而跨 PCIe 的搬运会抹掉 GPU 的全部优势。

此外，“Mandelbrot 内核受依赖链延迟限制” 这一发现把 SMT、ILP 和 GPU 的延迟隐藏串到了一起。同一段代码在不同硬件上的加速比差别巨大，很大程度上就取决于硬件有多少手段去填满依赖链留下的空闲周期。

# 参考文献

[1] K. Fatahalian, K. Olukotun. CS149: Parallel Computing, Stanford University, Fall 2024. `https://gfxcourses.stanford.edu/cs149/fall24`.

[2] M. Pharr, W. R. Mark. ispc: A SPMD Compiler for High-Performance CPU Programming. In *Innovative Parallel Computing (InPar)*, 2012.

[3] S. Williams, A. Waterman, D. Patterson. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Communications of the ACM*, 52(4), 2009.

[4] G. M. Amdahl. Validity of the Single Processor Approach to Achieving Large Scale Computing Capabilities. In *AFIPS Spring Joint Computer Conference*, 1967.

[5] R. D. Blumofe, C. E. Leiserson. Scheduling Multithreaded Computations by Work Stealing. *Journal of the ACM*, 46(5), 1999.

[6] D. M. Tullsen, S. J. Eggers, H. M. Levy. Simultaneous Multithreading: Maximizing On-Chip Parallelism. In *ISCA*, 1995.

[7] J. D. McCalpin. Memory Bandwidth and Machine Balance in Current High Performance Computers. *IEEE TCCA Newsletter*, 1995.

[8] U. Drepper. What Every Programmer Should Know About Memory. Red Hat, 2007.

[9] E. Lindholm, J. Nickolls, S. Oberman, J. Montrym. NVIDIA Tesla: A Unified Graphics and Computing Architecture. *IEEE Micro*, 28(2), 2008.

[10] V. Volkov. Better Performance at Lower Occupancy. GPU Technology Conference, 2010.

[11] J. L. Hennessy, D. A. Patterson. *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[12] S. Lloyd. Least Squares Quantization in PCM. *IEEE Transactions on Information Theory*, 28(2), 1982.

[13] Intel. Intel 64 and IA-32 Architectures Optimization Reference Manual, 2023.